BAB III

ENCAPSULATION

A. TUJUAN

1. Mahasiswa mampu menjelaskan konsep, tujuan, dan fungsi *encapsulation* dalam pemrograman berorientasi objek.
2. Mahasiswa mampu menerapkan konsep *encapsulation* dalam *class.*

B. ALAT DAN BAHAN

1. Laptop
2. Mouse
3. IDE Java (Netbeans 14)

C. KESEHATAN DAN KESELAMATAN KERJA

1. Hati-hatilah dalam memakai perangkat elektronik
2. Pastikan kabel listrik terpasang dan dalam kondisi baik
3. Lakukan praktikum dalam posisi duduk yang benar
4. Jauhkan kabel listrik dari sentuhan anda
5. Gunakan alas kaki celana Panjang dan kemeja
6. Gunakan kacamata anti radiasi layar

D. TEORI

1. Enkapsulasi dan *Modifier*

Enkapsulasi merupakan teknik yang membuat variabel/field class menjadi bersifat private dan menyediakan akses ke variabel/field melalui public method. Jika field di deklarasikan sebagai private, maka field ini tidak bisa diakses oleh siapapun diluar class, dengan demikian field disembunyikan di dalam class.

Manfaat utama teknik enkapsulasi adalah kita mampu memodifikasi kode tanpa merusak kode yang telah digunakan pada class lain. Enkapsulasi memiliki manfaat sebagai berikut:

- Modularitas

  Source code dari sebuah class dapat dikelola secara independen dari source code class yang lain. Perubahan internal pada sebuah class tidak akan berpengaruh bagi class yang menggunakannya.
- Information Hiding

Penyembunyian informasi yang tidak perlu diketahui objek lain. jika Anda ingin beberapa atribut hanya dapat diubah hanya dengan method tertentu, tentu Anda ingin menyembunyikannya dari obyek lain pada class. Di Java, implementasi tersebut disebut dengan access modifiers.

2. Penerapan enkapsulasi dalam class

Kita dapat menyembunyikan informasi dari suatu *class* sehingga anggota-anggota *class* tersebut dapat diakses dari luar. Adapaun caranya adalah cukup dengan memberikan akses *control private* Ketika mendeklarasikan suatu atribut atau *method.* Contoh :

```
private int nim;
```

encapsulation adalah suatu cara untuk menyembunyikan implementasi detail dari suatu class. Enkapsulasi mempunyai dua hal mendasar, yaitu :

1) *Information hiding*
2) Menyediakan suatu perantara (method) untuk pengaksesan data.

Contoh 3.1. Method

```
public class mahasiswa {
      private int nim;
      public void setNim(int n) {
         nim=n;
       }
}
```

`Constructor` adalah suatu *method* yang pertama kali dijalankan pada saat pembuatan suatu objek. Konstruktor mempunyai ciri yaitu :

- Mempunyai nama yang sama dengan nama class,
- Tidak mempunyai return type (seperti void, int, double, dan lain-lain).

Contoh 3.2. Konstruktor

```
public class mahasiswa {
      private int nim; private String nama;
      public mahasiswa(int n, String m) {
      nim=n; nama=m;
    }
}
```

Suatu *class* dapat mempunyai lebih dari 1 konstruktor dengan syarat daftar parameternya tidak boleh ada yang sama.

Contoh 3.3. Konstruktor

```
public class mahasiswa {
private int nim;
private String nama;
public mahasiswa(String m) {
      nim=0;
      nama="";
}
public mahasiswa(int n, String m) {
     nim=n; nama=m;
     }
}
```

Terdapat 4 macam access modifiers di JAVA, yaitu : *public, private, protected* dan *default.* 3 tipe akses pertama tertulis secara ekplisit pada kode untuk mengindikasikan tipe akses, sedangkan yang keempat yang merupakan tipe *default*, tidak diperlukan penulisan *keyword* atas tipe.

Tabel 3.1. Tipe Akses Modifier

No.	Modifier	Class	Paket Sama		Paket Berbeda	
			Extend	Instan	Extend	Instan
1	Public	√	√	√	√	√
2	Protected	√	√	√	√	
3	Default	√	√	√		
4	Private	√				

Public

Dapat dilihat pada table diatas bahwa keyword Public dapat diakses didalam class itu sendiri, dapat diakses dengan menggunakan metode *extend* dan instan pada paket yang sama, serta dapat diakses dengan metode *extend* maupun instan dalam paket yang berbeda. Artinya hak akses *public* dapat diakses oleh sembarang object manapun dan dimanapun posisinya serta dengan apapun caranya. Data maupun *method* yang bersifat

public dapat diakses oleh semua bagian didalam program. Untuk mendeklarasikan suatu data atau method dengan tingkat akses *public*, gunakan kata kunci *public*.

Protected

Suatu data maupun *method* yang dideklarasikan dengan tingkat akses *protected* dapat diakses oleh kelas yang memilkinya dan juga oleh kelas–kelas yang masih memiliki oleh hubungan turunan. Sebagai contoh, apabila data x dalam kelas A dideklarasikan sebagai *protected*, maka kelas B (yang merupakan turunan dari kelas A) diizinkan untuk mengakses data x. Namun apabila terdapat kelas lain, misalnya C (yang bukan merupakan turunan dari kelas A maupun B), tetap tidak dapat mengakses data – data yang dideklarasikan dengan tingkat akses *protected*. Untuk mendeklarasikan suatu data atau *method* dengan tingkat akses *protected*, gunakan kata kunci *ptotected*

Private

Dengan mendeklarasikan data dan method menggunakan tingkat akses private, maka data dan *method* tersebut hanya dapat diakses oleh kelas yang memilikinya saja. Ini berarti data dan *method* tersebut tidak boleh diakses atau digunakan oleh kelas-kelas lain yang terdapat didalam program. Untuk mendeklarasikan suatu data atau *method* dengan tingkat akses *private*, gunakan kata kunci *private.*

Contoh 3.4.

```
public class mahasiswa
{
      private String nama; //akses dasar terhadap variabel
      private String getNama() //akses dasar terhadap metode
{
        return nama;
   }
}
```

Pada contoh diatas, variabel nama dan *method* getNama() hanya dapat diakses oleh *method internal class* tersebut.

Default

Untuk hak akses *default*, sebenarnya hanya ditujukan untuk *class* yang ada dalam satu

paket, atau istilahnya hak akses yang berlaku untuk satu folder saja (tidak berlaku untuk *class* yang tidak satu folder/package)

Contoh 3.5.

```
public class mahasiswa{
    String nama; //akses dasar terhadap variabel
    String getNama(){ //akses dasar terhadap method
        return nama;
    }
}
```

Pada contoh diatas, variabel nama dan *method* getNama() hanya dapat diakses oleh *method internal class* tersebut.

Tabel 3.2 Karakteristik Access Modifier

Modifier	Class dan Interface	Method dan Variabel
Default (tidak ada modifier) Friendly	Dikenali dipaketnya	Diwarisi subclass di paket yang sama dengan superclassnya. Dapat diakses oleh method method di class-class yang sepaket.
Public	Dikenali dimanapun	Diwarisi oleh semua subclassnya. Dapat diakses dimanapun.
Protected	Tidak dapat diterapkan	Diwarisi oleh semua subclassnya. Dapat diakses oleh method-method di class class yang sepaket.
Private	Tidak dapat diterapkan	Tidak diwarisi oleh subclassnya Tidak dapat diakses oleh class lain.

This

Suatu besaran referensi khusus yang digunakan di dalam method yang dirujuk untuk objek yang sedang belaku. '`This`' digunakan ketika nama atribut yang sama dengan nama variable lokal.

Contoh 3.6.

```
public class Lagu {
    private String band;
    private String judul;
    public void IsiParam(String judul,String band) {
        this.judul = judul;
        this.band = band;
```

```
    }
    public void cetakKeLayar() {
       if(judul==null && band==null) return;
    System.out.println("Judul : " + judul +"\nBand : " + pencipta);
    }
}
public class DemoLagu {
    public static void main(String[] args) {
        Lagu song = new Lagu();
        song.IsiParam("Dance Beside","All American Reject ");
        song.cetakKeLayar();
    }
}
```

Output :

```
Judul : Dance Beside
Pencipta: All American Reject
```

E. PRAKTIKUM

Praktikum 3.1.

```
1   package praktikum4;
2
3   class atas
4   {
5       public int a;
6       protected int b;
7       private int c;
8   }
9   public class Encapsulasi1 {
10      public static void main(String[] args){
11          atas objek = new atas();
12          objek.a = 2;
13          objek.b = 3;
14          System.out.println("Nilai a : " + objek.a);
15          System.out.println("Nilai a : " + objek.b);
16      }
17  }
```

Praktikum 3.2.

```
1   package praktikum4;
2
3   class atas
4   {
5       public int a;
6       protected int b;
7       private String alamat;
8
9       public String getAlamat(){
10          return alamat;
11      }
12      public void setAlamat (String tempString){
13          alamat = tempString;
14      }
15  }
16  public class Encapsulasi2 {
17      public static void main(String[] args){
18          atas objek = new atas();
19          objek.a = 2;
20          objek.b = 3;
21          objek.setAlamat("Makassar");
22          System.out.println("Nilai a : " + objek.a);
23          System.out.println("Nilai a : " + objek.b);
24          System.out.println("Alamat : " + objek.getAlamat());
25      }
26  }
```

Praktikum 3.3.

```java
package praktikum4;

public class enkapsulasi {
    private int alas, tinggi;
    private double luasSegitiga;

    public void setAlas(int alas){
        this.alas = alas;
    }
    public int getAlas(){
        return alas;
    }
    public void setTinggi(int tinggi){
        this.tinggi = tinggi;
    }
    public int getTinggi(){
        return tinggi;
    }
    public void setLuasSegitiga(int alas, int tinggi){
        luasSegitiga = 0.5 * (double)(alas * tinggi);
    }
    public double getLuasSegitiga(){
        return luasSegitiga;
    }

}
```

```java
package praktikum4;

public class MainEnkapsulasi {
    public static void main (String args[])
    {
        enkapsulasi ob = new enkapsulasi();
        ob.setAlas(5);
        ob.setTinggi(7);
        System.out.println("Alas Segitiga : " + ob.getAlas());
        System.out.println("Tinggi Segitiga : " + ob.getTinggi());
        ob.setLuasSegitiga(ob.getAlas(), ob.getTinggi());
        System.out.println("Luas Segitiga : " + ob.getLuasSegitiga());
    }
}
```

F. SOAL TANTANGAN

1. Buatlah program enkapsulasi sederhana yang dapat menampilkan biodata Anda, berupa : nama, nim, jurusan, fakultas, universitas, alamat, email, hobi, dan keahlian.